

# Statistical Programming Languages

## Data frames

Katarzyna Reluga<sup>1</sup>

---

<sup>1</sup>Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

## Factor variables (1)

- ▶ Class factor to store categorical variables
- ▶ Advantages
  - ▶ Implemented correctly for statistical modeling
  - ▶ Very useful in many different types of graphics
  - ▶ More efficient use of memory compared to character
- ▶ Factors in R are stored as a vector of integer values!

```
f <- c(1, 2, 2, 1, 1, 2, 1, 1, 1, 2, 2, 2, 1)
class(f)
## [1] "numeric"
f <- as.factor(f)
class(f)
## [1] "factor"
f
## [1] 1 2 2 1 1 2 1 1 1 2 2 2 1
## Levels: 1 2
```

## Factor variables (2)

```
f.labels <- factor(f, labels = c("m", "f"))
f.labels
## [1] m f f m m f m m m f f f m
## Levels: m f
str(f.labels) # str() displays objects' structure
## Factor w/ 2 levels "m","f": 1 2 2 1 1 2 1 1 1 2 ...
# R stores factors as a vector of integers with labels.

# Alternative way to create a factor
myfactor <- factor(c("smoker", "non-smoker",
                     "non-smoker", "smoker",
                     "non-smoker", "smoker",
                     "smoker", "non-smoker"))
myfactor
## [1] smoker      non-smoker non-smoker smoker
## [5] non-smoker smoker      smoker      non-smoker
## Levels: non-smoker smoker
```

# Data frames

- ▶ Matrices can only contain one datatype
- ▶ Analyses often require tabular data of different types
- ▶ Data frames can contain several data types; Usually
  - ▶ each row corresponds to an observation
  - ▶ columns contain the variables
- ▶ Equal length of all elements (vectors) required

# Constructing data frames from vectors

```
# Age, numeric
a <- c(10, 20, 15, 43, 76, 41, 25, 46)
# Sex, factor
b <- factor(c("m", "f", "m", "f", "m", "f", "m", "f"))
# Siblings, numeric
c <- c(2, 3, 0, 1, 2, 1, 0, 1)
# Create data frame
mydf <- data.frame(a, b, c)
names(mydf) <- c("Age", "Sex", "Siblings")
head(mydf, 3)
##    Age Sex Siblings
## 1  10  m         2
## 2  20  f         3
## 3  15  m         0
```

## Constructing data frames from matrices

```
A <- matrix(1:12, ncol = 4); A
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
class(A)
## [1] "matrix" "array"
A.df <- as.data.frame(A); A.df
##    V1 V2 V3 V4
## 1   1  1  4  7 10
## 2   2  2  5  8 11
## 3   3  3  6  9 12
class(A.df)
## [1] "data.frame"
```

# Accessing data frame elements (1)

- ▶ Access elements of a data frame through brackets [ , ] (like matrices)
- ▶ Access a column of a data frame by its name through \$ (like lists)

```
mydf[, 1]
## [1] 10 20 15 43 76 41 25 46
mydf$Age
## [1] 10 20 15 43 76 41 25 46
mydf[, "Age"]
## [1] 10 20 15 43 76 41 25 46
mydf[3, 3]
## [1] 0
mydf[3, 3] <- 2 # change value
mydf[3, 3]
## [1] 2
```

## Accessing data frame elements (2)

```
mydf[-2, c("Age", "Sex")]      # all rows except 2nd
##   Age Sex
## 1  10  m
## 3  15  m
## 4  43  f
## 5  76  m
## 6  41  f
## 7  25  m
## 8  46  f
```



## Accessing data frame elements (3)

Attaching the data frame allows to access variables directly (without repeating the name of the data frame).

```
attach(mydf)
Age
## [1] 10 20 15 43 76 41 25 46
Sex
## [1] m f m f m f m f
## Levels: f m
Siblings
## [1] 2 3 2 1 2 1 0 1
detach(mydf)
Age
## Error: object 'Age' not found
```

## Accessing data frame elements (4)

```
# males over 30
mydf[(mydf$Sex == "m") & (mydf$Age > 30), ]
##   Age Sex Siblings
## 5  76  m         2
# male or over 30
mydf[(mydf$Sex == "m") | (mydf$Age > 30), ]
##   Age Sex Siblings
## 1  10  m         2
## 3  15  m         2
## 4  43  f         1
## 5  76  m         2
## 6  41  f         1
## 7  25  m         0
## 8  46  f         1
```

## Adding variables

```
head(mydf, 3)
##    Age Sex Siblings
## 1  10  m         2
## 2  20  f         3
## 3  15  m         2
# add Income to the data frame
mydf$Income <- c(1700, 2100, 2300, 2050, 2800, 1450,
                 3400, 2000)
head(mydf, 3)
##    Age Sex Siblings Income
## 1  10  m         2   1700
## 2  20  f         3   2100
## 3  15  m         2   2300
```

## Renaming and deleting variables

```
# rename variable
names(mydf)
## [1] "Age"      "Sex"      "Siblings" "Income"
names(mydf)[names(mydf) == "Income"] <- "IncomeUSD"
# or names(mydf)[4] <- "IncomeUSD"

# remove variable
mydf$Age <- NULL
head(mydf, 3)
##      Sex Siblings IncomeUSD
## 1    m         2      1700
## 2    f         3      2100
## 3    m         2      2300
```

## Sorting data frames (1)

```
# Sort by column "Sex"
mydf[order(mydf$Sex), ]
```

| ##   | Sex | Siblings | IncomeUSD |
|------|-----|----------|-----------|
| ## 2 | f   | 3        | 2100      |
| ## 4 | f   | 1        | 2050      |
| ## 6 | f   | 1        | 1450      |
| ## 8 | f   | 1        | 2000      |
| ## 1 | m   | 2        | 1700      |
| ## 3 | m   | 2        | 2300      |
| ## 5 | m   | 2        | 2800      |
| ## 7 | m   | 0        | 3400      |

## Sorting data frames (2)

```
# Sort by several columns
mydf[order(mydf$Sex, mydf$Siblings), ]
##      Sex Siblings IncomeUSD
## 4      f         1      2050
## 6      f         1      1450
## 8      f         1      2000
## 2      f         3      2100
## 7      m         0      3400
## 1      m         2      1700
## 3      m         2      2300
## 5      m         2      2800
```